GRAPHQL MIT RUBY ON RAILS

Schritt 33 - GraphQL Variablen, Input & Client Request

Project Management API - graphql-ruby - Axios - Devise

Lernziel: Klar unterscheiden, was zu GraphQL und was zu Rails gehört, Input-Namensregeln verstehen und einen vollständigen Client-Request mit Axios aufbauen.

1. Lernziele

- GraphQL Variables verstehen und nicht mit Rails-Parametern verwechseln.
- Scalar, Enum, Input Object und Relay-Mutation-Input unterscheiden.
- Verstehen, welche Input-Typen du selbst definierst und welche graphql-ruby erzeugt.
- Namenskonventionen für TaskInput, TaskUpdateInput und TaskFilterInput anwenden.
- camelCase im GraphQL-Client und snake_case in Ruby korrekt zuordnen.
- query + variables + operationName mit Axios an /graphql senden.
- Den gesamten Request-Flow vom Client bis zum Resolver nachvollziehen.

2. Was gehört zu GraphQL und was zu Rails?

Begriff	Ebene	Aufgabe
Variable	GraphQL	Dynamischer Wert einer Operation
Input Object	GraphQL	Strukturierter Eingabetyp
Enum	GraphQL	Erlaubte Werte begrenzen
Operation Name	GraphQL	Name einer Query oder Mutation
Types::Inputs::TaskInput	Rails / graphql-ruby	Ruby-Klasse, die einen GraphQL Input-Typ definiert
Resolver / Mutation	Rails / graphql-ruby	Verarbeitet validierte GraphQL-Daten
GraphqlController	Rails	Nimmt HTTP POST /graphql entgegen

Merksatz: Variables, Input Objects, Enums und Operation Names sind GraphQL-Konzepte. Rails/graphql-ruby implementiert und verarbeitet sie in Ruby.

3. GraphQL Variable

```
query GetProject($id: ID!) {
  project(id: $id) {
    id
    name
  }
}
```

- GetProject = Operation Name
- $id = GraphQL Variable
- ID! = verpflichtender GraphQL-Typ
- project(id: $id) = Variable wird an ein Field-Argument übergeben

Der Client sendet den tatsächlichen Wert getrennt im Objekt variables:

```
{
  "id": "10"
}
```

4. Was ist ein Input Object?

Bei vielen zusammengehörenden Eingabefeldern ist ein Input Object übersichtlicher als zahlreiche einzelne Argumente.

```
$input: CreateTaskInput!
```

```
{
  "input": {
    "projectId": "10",
    "attributes": {
```

```
      "title": "GraphQL lernen",
      "priority": "HIGH"
    }
  }
}
```

5. CreateTaskInput und TaskInput sind nicht dasselbe

5.1 TaskInput - von dir definierter Input-Typ

Datei: `app/graphql/types/inputs/task_input.rb`

```
module Types
  module Inputs
    class TaskInput < Types::BaseInputObject
      argument :title, String, required: true
      argument :description, String, required: false
      argument :priority, Types::Enums::TaskPriorityEnum, required: false
      argument :due_date, GraphQL::Types::ISO8601Date, required: false
      argument :assignee_id, ID, required: false
    end
  end
end
```

Im GraphQL-Schema sieht der Client ungefähr:

```
input TaskInput {
  title: String!
  description: String
  priority: TaskPriorityEnum
  dueDate: ISO8601Date
  assigneeId: ID
}
```

5.2 CreateTaskInput - Wrapper der Relay-Mutation

```
class BaseMutation < GraphQL::Schema::RelayClassicMutation
end
```

```
class CreateTask < BaseMutation
  argument :project_id, ID, required: true
  argument :attributes, Types::Inputs::TaskInput, required: true
end
```

graphql-ruby exponiert daraus ungefähr:

```
input CreateTaskInput {
  projectId: ID!
  attributes: TaskInput!
  clientMutationId: String
}
```

```
input
├── projectId
└── attributes
    ├── title
    ├── description
    ├── priority
    ├── dueDate
    └── assigneeId
```

6. Namenskonventionen für Input-Typen

Name	Verwendung	Bedeutung
TaskInput	Create	Daten zum Erstellen eines Tasks
TaskUpdateInput	Update	Änderbare Task-Felder
TaskFilterInput	Query	Filterbedingungen für Tasks

```
Create  -> ResourceInput
Update  -> ResourceUpdateInput
Filter  -> ResourceFilterInput
```

Vermeide unklare Namen wie TaskData oder TaskParams, wenn der Zweck des Inputs dadurch nicht eindeutig wird.

7. Warum Create und Update getrennte Inputs brauchen

```
class TaskInput < Types::BaseInputObject
  argument :title, String, required: true
end
```

Beim Erstellen ist `title` verpflichtend. Ein Update ist dagegen normalerweise partiell:

```
class TaskUpdateInput < Types::BaseInputObject
  argument :title, String, required: false
  argument :status, Types::Enums::TaskStatusEnum, required: false
end
```

```
{
  "status": "DONE"
}
```

8. camelCase im Client - snake_case in Ruby

GraphQL	Ruby
projectId	project_id
assigneeId	assignee_id
dueDate	due_date
dueFrom	due_from
sortBy	sort_by
sortDirection	sort_direction

9. Vollständige GetTasks Query

```
query GetTasks(
  $projectId: ID!
  $filter: TaskFilterInput
  $sortBy: TaskSortFieldEnum
  $sortDirection: SortDirectionEnum
  $first: Int!
  $after: String
) {
  tasks(
    projectId: $projectId
    filter: $filter
    sortBy: $sortBy
    sortDirection: $sortDirection
    first: $first
    after: $after
  ) {
    nodes {
      id
      title
      status
      priority
      dueDate
    }

    pageInfo {
      hasNextPage
      endCursor
```

```
    }
  }
}
```

10. Client-Request mit Axios

```
import axios from "axios";

const query = `
  query GetTasks(
    $projectId: ID!
    $filter: TaskFilterInput
    $sortBy: TaskSortFieldEnum
    $sortDirection: SortDirectionEnum
    $first: Int!
    $after: String
  ) {
    tasks(
      projectId: $projectId
      filter: $filter
      sortBy: $sortBy
      sortDirection: $sortDirection
      first: $first
      after: $after
    ) {
      nodes {
        id
        title
        status
        priority
        dueDate
      }
      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
`;

const variables = {
  projectId: "10",
  filter: {
    status: ["TODO", "IN_PROGRESS"],
    priority: ["HIGH"],
    search: "graphql"
  },
  sortBy: "DUE_DATE",
  sortDirection: "ASC",
  first: 20,
  after: null
};

const response = await axios.post(
  "http://localhost:3000/graphql",
  {
    query,
    variables,
    operationName: "GetTasks"
  },
  {
    withCredentials: true
  }
);
```

Devise: Wenn deine Authentifizierung über Session-Cookies läuft, ist `withCredentials: true` bei getrennten Origins wichtig. CORS/CSRF muss dazu passend konfiguriert sein.

11. HTTP-Body

```
{
  "query": "query GetTasks(...) { ... }",
  "variables": {
    "projectId": "10",
    "filter": {
      "status": ["TODO", "IN_PROGRESS"],
      "priority": ["HIGH"],
      "search": "graphql"
    },
    "sortBy": "DUE_DATE",
    "sortDirection": "ASC",
    "first": 20,
    "after": null
  },
  "operationName": "GetTasks"
}
```

12. Rails empfängt den Request

Datei: app/controllers/graphql_controller.rb

```
def execute
  variables = prepare_variables(params[:variables])
  query = params[:query]
  operation_name = params[:operationName]

  context = {
    current_user: current_user
  }

  result = YourAppSchema.execute(
    query,
    variables: variables,
    context: context,
    operation_name: operation_name
  )

  render json: result
end
```

13. Mapping in den Resolver

```
argument :project_id, ID, required: true
argument :filter, Types::Inputs::TaskFilterInput, required: false
argument :sort_by, Types::Enums::TaskSortFieldEnum, required: false

def resolve(project_id:, filter: nil, sort_by:)
  # project_id => "10"
  # filter[:status] => ["todo"]
  # sort_by => "due_date"
end
```

14. Enum Mapping

```
# Client:
{
  "sortBy": "DUE_DATE",
  "sortDirection": "ASC"
}

# Ruby Enum:
value "DUE_DATE", value: "due_date"
value "ASC", value: "asc"

# Resolver:
sort_by        # => "due_date"
sort_direction # => "asc"
```

Wichtig: Der Client sendet die GraphQL-Enum-Namen aus dem Schema, also z. B. DUE_DATE und ASC.

15. CreateTask Mutation - kompletter Request

```
mutation CreateTask($input: CreateTaskInput!) {
  createTask(input: $input) {
    task {
      id
      title
      status
      priority
    }
    errors
  }
}
```

```
{
  "input": {
    "projectId": "10",
    "attributes": {
      "title": "GraphQL Variables lernen",
      "description": "Schritt 33",
      "priority": "HIGH",
      "dueDate": "2026-08-25",
      "assigneeId": "20"
    }
  }
}
```

16. Wann braucht man ein Input Object?

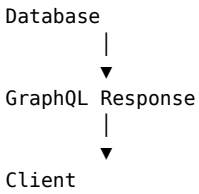
Für ein oder zwei einfache Argumente ist ein direktes Argument völlig ausreichend:

```
project(id: $id)
```

Viele Felder mit demselben Zweck sollten dagegen gruppiert werden, z. B. als `TaskFilterInput`.

17. Gesamter Request-Flow

```
CLIENT
│
├── query
├── variables
└── operationName
        │
        ▼
POST /graphql
        │
        ▼
GraphqlController
        │
        ▼
Schema.execute
        │
        ▼
GraphQL Validation
        │
        ├── Variable Types
        ├── Input Types
        └── Enum Values
        │
        ▼
Resolver / Mutation
        │
        ▼
Policy / Service / Model
        │
        ▼
```



18. Häufige Fehler

- Der Client sendet "due_date" statt "dueDate".
- Der Client sendet "due_date" oder "asc" statt der GraphQL-Enums "DUE_DATE" / "ASC".
- TaskInput wird gleichzeitig für Create, Update und Filter verwendet.
- projectId wird hart in die Query geschrieben statt über $projectId übergeben.
- CreateTaskInput und TaskInput werden verwechselt.
- operationName wird mit dem Namen einer Ruby-Klasse verwechselt.
- withCredentials: true wird bei Devise-Session-Cookies vergessen.
- Der Client sendet currentUserId, obwohl die Identität serverseitig aus Devise current_user kommen sollte.

19. Übung

1. GetTasks mit $projectId, $filter, $sortBy, $sortDirection, $first und $after schreiben.
2. GetTasks mit Axios aufrufen und response.data.data.tasks.nodes ausgeben.
3. Nur über variables von CREATED_AT auf DUE_DATE umstellen.
4. TODO + IN_PROGRESS filtern, ohne den Query-String zu verändern.
5. CreateTask mit $input: CreateTaskInput! schreiben.
6. Im Rails-Log prüfen, welche Werte im Resolver ankommen.

20. Zusammenfassung

Der wichtigste Satz: Der Client kennt nur das GraphQL-Schema. Er sendet `query + variables + operationName`. graphql-ruby validiert und konvertiert die Daten und ruft danach Resolver oder Mutation auf.

```
GraphQL Client          Ruby

projectId        ->     project_id
assigneeId       ->     assignee_id
dueDate          ->     due_date

TaskInput        ->     Create-Daten
TaskUpdateInput  ->     Update-Daten
TaskFilterInput  ->     Query-Filter

CreateTask       ->     Mutation / Operation
CreateTaskInput  ->     Relay-Mutation-Wrapper
```